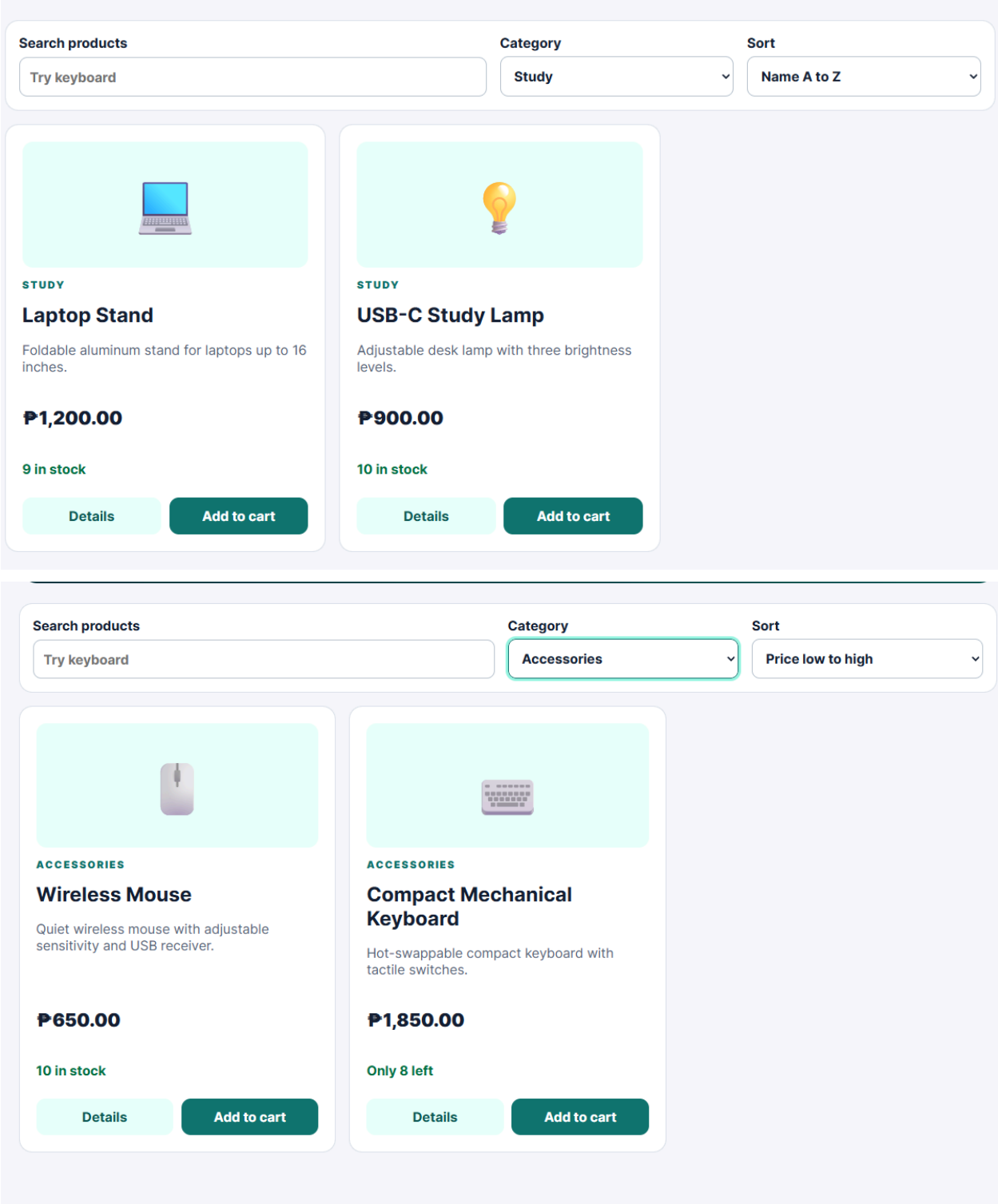
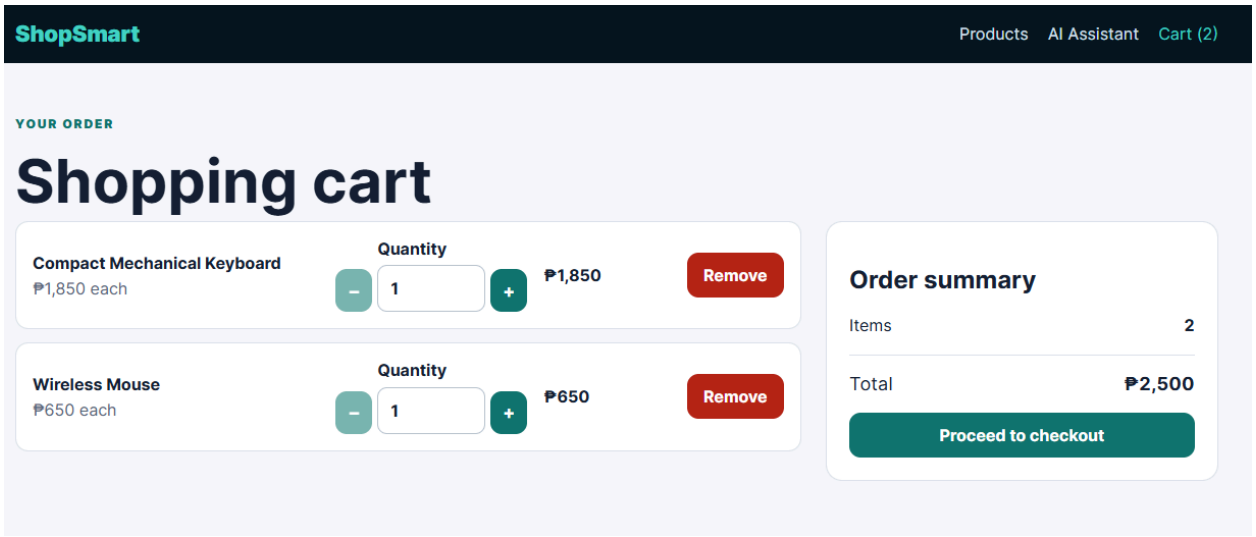
Name: Andrik Brenth E. Canlas Section: BSIT 4 - A

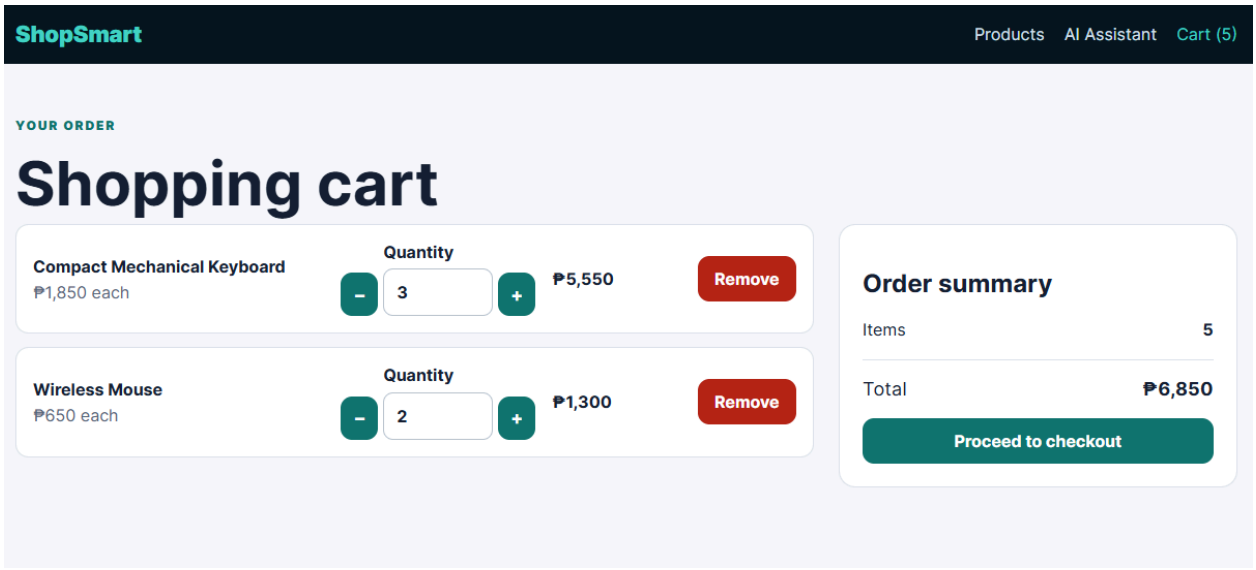
1. Repository link: https://github.com/canlasandrikbrenth-source/Canlas-ShopSmart

Screenshots:
1. Filtered and Sorted Catalog

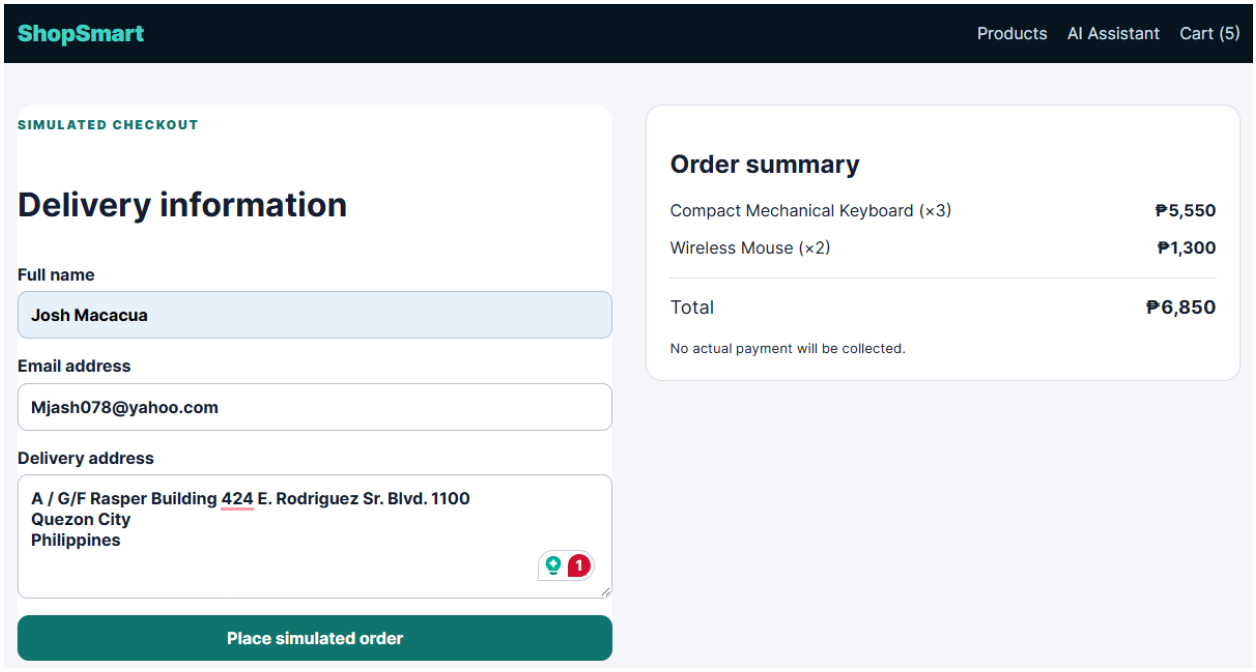


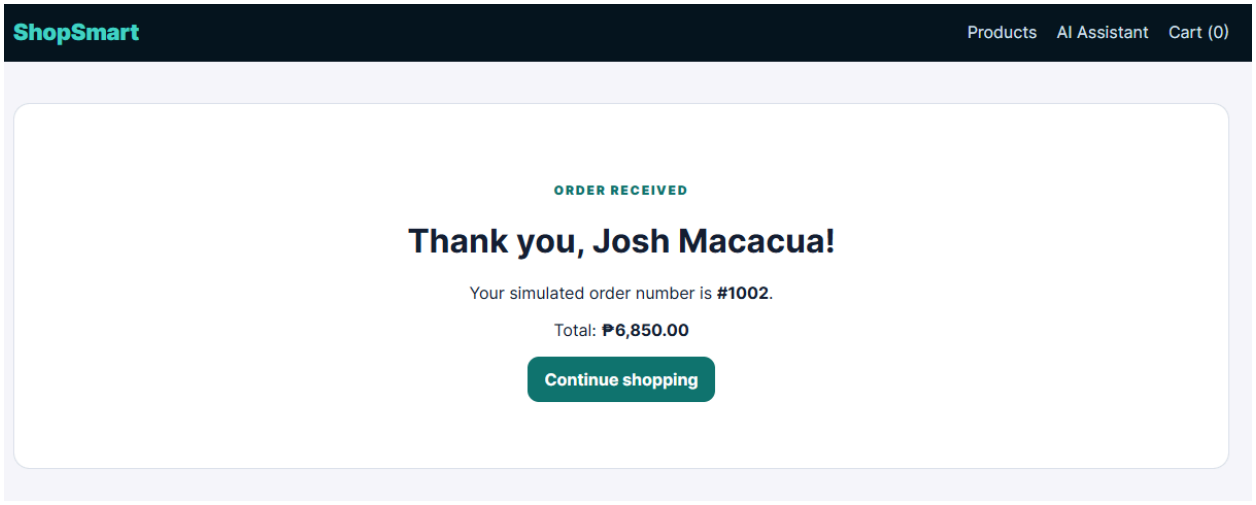
2. Cart with Two Products and Changed Quantity





3. Checkout Review with Line Items and Total





Npm run check output:
> shopsmart-starter@1.0.0 check
> npm run lint --workspace frontend && npm run lint --workspace backend && npm run test && npm run build

> shopsmart-frontend@1.0.0 lint
> eslint .

> shopsmart-backend@1.0.0 lint
> eslint .

> shopsmart-frontend@1.0.0 test
> vitest run
✓ src/utils/cart.test.js (7 tests) 3ms
Test Files 1 passed (1)
Tests 7 passed (7)

> shopsmart-backend@1.0.0 test
> node --test
✔ parseBudget extracts a peso budget
✔ rankProducts respects the budget
✔ assistant response contains traceable sources
✔ a valid classroom order passes validation
✔ duplicate products fail order validation
✔ an oversized assistant question fails validation
i tests 6
i suites 0
i pass 6
i fail 0

> shopsmart-frontend@1.0.0 build
> vite build
✓ built in 1.20s

Explanation:

In ProductsPage.jsx, the user's sort preference is tracked with a simple sortOption state. Sorting and filtering run as a safe, non-mutating pipeline: search and category filters first output visibleProducts, and useMemo sorts a shallow copy of that result so the original data stays untouched.

For cart updates, the + and - buttons reuse the setQuantity handler from CartContext. This routes everything through the shared validation logic in cart.js rather than creating separate rules. As a result, the input box, header badge, item subtotals, and overall total stay perfectly in sync while enforcing min/max stock limits.

Finally, the checkout review pulls its items, quantities, and line subtotals directly from useCart() and calculateItemSubtotal(), so the itemized breakdown always matches the final total due.